SINGAPORE MANAGEMENT UNIVERSITY

School of Computing and Information Systems (SCIS)

SE301 - Software Engineering & Object-Oriented Design

Singapore Vertical Farming Research Management System (SVFRMS)

Legacy System Analysis, Code Smells & SOLID Refactoring,
Thread-Safe Concurrency, and Extensible Software Architecture

PROJECT GROUP: GROUP G1

No.	Team Member Name	Role & Focus Area
1	Keane (Team Member 1)	Architecture, SOLID & Concurrency Lead
2	Team Member 2	Domain Modeling & Sensor Subsystem
3	Team Member 3	Control Subsystem & Design Patterns
4	Team Member 4	Reporting, Verification & Documentation

Submission Archive Contents & Compliance:

- src/ & pom.xml: Complete refactored Java source code in cohesive packages (org.example.*).
- run.jar: Standalone executable JAR (java -jar run.jar) with Main-Class: org.example.SVFRMS.
- README.pdf: Technical Report (6 content pages, strictly within the 3-6 page limit excluding cover).
- Behavior Preservation: 100% faithful to legacy SVFRMS.java functionality, console logs, and archive output.

1. LEGACY SYSTEM ANALYSIS

The Singapore Vertical Farming Research Management System (SVFRMS) monitors and regulates indoor vertical farming growth chambers cultivating specialty crops (Tomato, Strawberry, Lettuce, Spinach, Chili, and Cucumber). Its purpose is to ingest real-time environmental telemetry across five sensor types (Temperature, Humidity, Soil, Light, and Nutrient), detect crop-specific threshold violations, dispatch automated mitigation requests to six actuators (Cooling Fan, Water Pump, Nutrient Injector, Humidifier, Dehumidifier, and Grow Light), track facility-wide water and energy usage, and generate daily console and file reports (archive/daily_report.txt).

In the legacy implementation, the entire system is written inside a single 393-line file (org.example.SVFRMS) containing one outer class and four static inner classes:

- Global State Container (SVFRMS, lines 6-31): Declares 19 mutable public static fields storing active/archived alerts, sensorLogs, reports, per-chamber maps (chamberStates, chamberSummaries), resource counters (totalWaterUsage, totalEnergyUsage), a bounded controlQueue of strings, and four lock objects (reportLock, alertLock, waterPumpLock, nutrientLock).
- Passive Inner Models (ChamberState & ChamberSummary, lines 33-46): Static inner classes with package-private mutable fields tracking chamber readings (temperature=24, humidity=70, soilMoisture=60, lightLevel=600, nutrientLevel=70) and per-chamber alert/action counts.
- Concurrent Threads (ChamberThread & ControlThread, lines 251-295, 349-392): Four ChamberThreads (CH1 Tomato, CH2 Strawberry, CH3 Lettuce, CH4 Chili) loop 20 times generating random readings via shared static Random, invoking processSensor(), and incrementing totalWaterUsage += 2 and totalEnergyUsage += 1. ControlThread dequeues 'chamber|control' strings from controlQueue and invokes activateControl(). At shutdown, two threads run archiveAlerts() and generateDailyReport() concurrently, followed by saveArchiveReport().

2. CODE ANALYSIS AND LIMITATIONS (LEGACY CODE -> ISSUE -> FIX)

Issue 1: God Class, Public Static Mutable State & Broken Information Hiding (Lines 10-39)

SE301 Syllabus Category: Week 1 Generics | Week 2 Anti-Patterns (God Object) | Week 7 Pressman's Information Hiding & DIP

1. LEGACY CODE (SVFRMS.java)

```
public class SVFRMS {
    public static ArrayList<String> alerts = new ArrayList<>();
    public static HashMap<String, ChamberState> chamberStates = new HashMap<>();
    public static int totalWaterUsage = 0; public static int totalEnergyUsage = 0;
    public static Object reportLock = new Object(); public static Object waterPumpLock = new Object(); // unused
    static class ChamberState { double temperature = 24; double humidity = 70; ... }
}
```

2. Issues with the Legacy Code:

- God Object Anti-Pattern & Low Cohesion (Week 2 & Week 7): All 393 lines reside in SVFRMS.java, mixing domain state, sensor evaluation, device control, thread queuing, logging, and file I/O in one class (high LCOM, high CBO).
- Broken Information Hiding & DIP (Week 7): All 19 fields in SVFRMS are public static mutable and typed to concrete classes (ArrayList, HashMap) instead of interfaces (List, Map). ChamberState fields have no encapsulation.

3. How It Was Fixed:

Extracted ChamberState into org.example.model with private fields and a ReentrantReadWriteLock (Week 10), removed unused locks (waterPumpLock, nutrientLock), and encapsulated state inside focused services wired by the SVFRMS Facade:

REFACTORED CODE (How It Was Fixed)

```
public class ChamberState {
    private double temperature = 24; private double humidity = 70;
    private final ReentrantReadWriteLock rwLock = new ReentrantReadWriteLock();
    private final Lock readLock = rwLock.readLock(), writeLock = rwLock.writeLock();
    public void setTemperature(double t) { writeLock.lock(); try { temperature = t; } finally { writeLock.unlock(); } }
}
```

Issue 2: Selector / Flag Arguments & SRP/ISP Violation in handleEvent() (Lines 62-88)

SE301 Syllabus Category: Week 7 Code Smells (Selector / Flag Arguments) | Week 7 SOLID (SRP & ISP)

1. LEGACY CODE (SVFRMS.java)

```
public static void handleEvent(String type, String chamber, String crop, String message) {
    if (type.equals("LOG")) { sensorLogs.add(new Date() + " : " + message); }
    else if (type.equals("CONTROL_LOG")) { System.out.println("\t\u001B[32m[" + chamber + "][CONTROL] " + message + "\u001B[0m");
    else if (type.equals("CREATE_ALERT")) { alerts.add(message); totalAlerts++; totalAlertsGenerated++; ... }
    else if (type.equals("TRACK_ALERT")) { ChamberSummary s = getChamberSummary(chamber); s.alerts++; s.lastAlert = message; }
}
```

2. Issues with the Legacy Code:

- Selector / Flag Argument Code Smell (Week 7): The 'type' string flag forces one method to perform 6 unrelated operations ('LOG', 'ALERT_LOG', 'CONTROL_LOG', 'CREATE_ALERT', 'TRACK_ALERT', 'TRACK_ACTION').
- Unused Parameters & SRP/ISP Violation: Callers pass empty dummy strings ('') for unused parameters (e.g., handleEvent('LOG', '', '', msg)).

3. How It Was Fixed:

Removed handleEvent() and split its 6 flag branches into explicit, single-responsibility methods across EventLogger, AlertManager, and ChamberManager:

REFACTORED CODE (How It Was Fixed)

```
// EventLogger.java: dedicated logging methods (no selector flags or dummy arguments)
public void logSensor(String msg) { sensorLogs.add(new Date() + " : " + msg); }
public void logAlert(String ch, String crop, String msg) { System.out.println("["+ch+"]["+crop+"][ALERT] "+msg); }
public void logControl(String ch, String msg) { System.out.println("\t\u001B[32m["+ch+"][CONTROL] "+msg+"\u001B[0m"); }
```

2. Code Analysis and Limitations (Continued: DRY, Magic Numbers, Feature Envy, OCP)

Issue 3: DRY Violations, Magic Numbers, Side Effects & OCP in processSensor() (Lines 141-205)

SE301 Syllabus Category: Week 2 Anti-Patterns (Magic Numbers) & Strategy/Builder/Repository | Week 7 Code Smells (DRY, Side Effects) & OCP

1. LEGACY CODE (SVFRMS.java)

```
if (sensor.equals("Temperature")) {
    state.temperature = value;
    if (crop.equals("Tomato")) {
        if (value > 28) { handleEvent("TRACK_ALERT", chamber, "", chamber + " Tomato High Temp");
            handleEvent("CREATE_ALERT", ...); totalEnergyUsage += 10; addControlRequest(chamber, "Cooling Fan"); }
        if (value < 22) { handleEvent("TRACK_ALERT", ...); handleEvent("CREATE_ALERT", ...); totalEnergyUsage += 15; }
    } else if (crop.equals("Strawberry")) { if (value > 24) { ... } } // Repeated across 6 crops & 5 sensors!
}
```

2. Issues with the Legacy Code:

- Violation of DRY & Magic Numbers (Week 2 & Week 7): Repeats identical TRACK_ALERT and CREATE_ALERT calls across 19 conditional branches with hardcoded thresholds (28, 22, 24, 18, 80, 60, 45, 500, 40).
- Violation of Open-Closed Principle (OCP): Adding a new crop (e.g., Kale or Bell Pepper) or sensor type requires editing processSensor() in multiple places.
- Hidden Side Effects (Week 7): Evaluating a sensor reading secretly mutates global totalEnergyUsage and totalWaterUsage.

3. How It Was Fixed:

Encapsulated crop thresholds and resource increments inside immutable CropProfile objects (built via CropProfile.Builder and stored in CropRegistry). Replaced sensor branches with SensorProcessor strategies in SensorService and extracted recordAlert():

REFACTORED CODE (How It Was Fixed)

```
// SensorService.java: evaluates any CropProfile polymorphically without crop-specific if-else chains (OCP & DRY)
registerProcessor(TEMPERATURE, (chamber, crop, value, state, profile) -> {
    state.setTemperature(value);
    if (profile == null || profile.getTemperatureRange() == null) return;
    ThresholdRange range = profile.getTemperatureRange();
    if (range.isAboveMax(value)) {
        recordAlert(chamber, crop, chamber + " " + crop + " High Temp");
        if (profile.getHighTempEnergyUsage() > 0) resourceMonitor.addEnergyUsage(profile.getHighTempEnergyUsage());
        controlService.addControlRequest(chamber, ControlService.COOLING_FAN);
    }
});
```

Issue 4: Feature Envy & Conditional Complexity in activateControl() (Lines 90-129)

SE301 Syllabus Category: Week 2 Strategy Pattern | Week 7 Code Smells (Feature Envy & DRY) | Week 7 SOLID (OCP & DIP)

1. LEGACY CODE (SVFRMS.java)

```
public static void activateControl(String chamber, String control) {
    ChamberState state = getChamberState(chamber);
    if (control.equals("Cooling Fan")) {
        state.temperature = Math.max(18, state.temperature - 5);
        handleEvent("TRACK_ACTION", chamber, "", "Cooling Fan Activated");
        handleEvent("CONTROL_LOG", chamber, "", "Cooling Fan Activated");
        handleEvent("CONTROL_LOG", chamber, "", "Temperature reduced to " + state.temperature + "C");
    } else if (control.equals("Water Pump")) { state.soilMoisture = Math.min(100, state.soilMoisture + 20); ... }
}
```

2. Issues with the Legacy Code:

- Feature Envy (Week 7 Code Smells): activateControl() reaches directly into ChamberState's internal fields (state.temperature, state.soilMoisture, etc.) to compute bounded adjustments instead of delegating to ChamberState.
- OCP & DRY Violation: Hardcodes a 6-branch if-else chain over device names and duplicates TRACK_ACTION and CONTROL_LOG calls 6 times.

3. How It Was Fixed:

Moved bounded state adjustments into thread-safe methods on ChamberState (adjustTemperature, adjustSoilMoisture, etc.) and replaced the 6-branch conditional with ControlDevice strategies registered in ControlService:

REFACTORED CODE (How It Was Fixed)

```
// ChamberState.java: owns its data and bounded adjustment logic (eliminates Feature Envy)
public double adjustTemperature(double delta, double minBound, double maxBound) {
    writeLock.lock();
    try { this.temperature = Math.min(maxBound, Math.max(minBound, this.temperature + delta)); return temperature; }
    finally { writeLock.unlock(); }
}
// ControlService.java: dispatches to ControlDevice strategy and logs/tracks once (OCP & DRY)
ControlDevice device = devices.get(controlName);
if (device != null) {
    String detailMsg = device.applyAdjustment(state);
    chamberManager.trackAction(chamberId, device.getActionName());
    eventLogger.logControl(chamberId, device.getActionName()); eventLogger.logControl(chamberId, detailMsg);
}
```

2. Code Analysis and Limitations (Continued: Multithreading, Race Conditions, Locks & Deadlocks)

Issue 5: Non-Atomic (++ / +=) Race Conditions & Inconsistent Alert Locking (Lines 72-86, 291-292)

SE301 Syllabus Category: Week 9 Race Conditions (3 Non-Atomic Micro-Steps) | Week 10 Explicit Locks & LockFactory

1. LEGACY CODE (SVFRMS.java)

```
// Executed concurrently by 4 ChamberThreads without holding any lock:
else if (type.equals("CREATE_ALERT")) { alerts.add(message); totalAlerts++; totalAlertsGenerated++; }
else if (type.equals("TRACK_ALERT"))  { ChamberSummary s = getChamberSummary(chamber); s.alerts++; s.lastAlert = message; }
// In ChamberThread.run() & processSensor():
totalWaterUsage += 2; totalEnergyUsage += 1; // and totalWaterUsage += 50; totalEnergyUsage += 10;
```

2. Issues with the Legacy Code:

- Non-Atomic Read-Modify-Write Race Condition (Week 9 Slides 9-14): Operations like totalAlerts++, summary.alerts++, and totalWaterUsage += k compile into 3 machine micro-steps (fetch from memory to register, add in register, write back to memory). Interleaving across 4 threads causes lost updates.
- Inconsistent Synchronization on alerts: archiveAlerts() synchronizes on alertLock, but CREATE_ALERT mutates alerts and totalAlerts with NO lock at all, and saveArchiveReport() reads all lists unlocked.

3. How It Was Fixed:

Implemented LockFactory.getLock(String) (Week 10 Singleton + Factory) to guard all alert operations under ALERT_LOCK in AlertManager, all water/energy updates under RESOURCE_LOCK in ResourceMonitor, and summary counters under ReentrantReadWriteLock in ChamberSummary:

REFACTORED CODE (How It Was Fixed)

```
// LockFactory.java (Week 10 Slides 19-20: Singleton + Factory Pattern)
public static Lock getLock(String lockName) {
    return INSTANCE.locks.computeIfAbsent(lockName, key -> new ReentrantLock());
}
// ResourceMonitor.java & AlertManager.java: explicit ReentrantLock in try / finally blocks
public void addWaterUsage(int liters) {
    resourceLock.lock(); try { totalWaterUsage += liters; } finally { resourceLock.unlock(); }
}
```

Issue 6: Check-Then-Act Race on Raw HashMap & Unsynchronized ArrayList (Lines 12, 48-64)

SE301 Syllabus Category: Week 9 Race Conditions (ConcurrentHashMap vs HashMap & Collections.synchronizedList, Slides 31-44)

1. LEGACY CODE (SVFRMS.java)

```
public static ArrayList<String> sensorLogs = new ArrayList<>();
public static HashMap<String, ChamberState> chamberStates = new HashMap<>();
public static ChamberState getChamberState(String chamber) {
    if (!chamberStates.containsKey(chamber)) { chamberStates.put(chamber, new ChamberState()); } // Check-Then-Act!
    return chamberStates.get(chamber);
}
```

2. Issues with the Legacy Code:

- Check-Then-Act Race Condition on Raw HashMap (Week 9 Slide 35): Two ChamberThreads calling containsKey(chamber) simultaneously can both see false and overwrite each other's ChamberState, or corrupt internal HashMap buckets.
- Unsynchronized ArrayList Mutation (Week 9 Slide 41): Four threads concurrently calling sensorLogs.add(...) on a raw ArrayList causes lost log entries or ArrayIndexOutOfBoundsException.

3. How It Was Fixed:

Replaced raw HashMap with ConcurrentHashMap and atomic computeIfAbsent() in ChamberManager, and wrapped sensorLogs with Collections.synchronizedList(new ArrayList<>()) in EventLogger:

REFACTORED CODE (How It Was Fixed)

```
// ChamberManager.java: atomic computeIfAbsent on ConcurrentHashMap eliminates Check-Then-Act race
private final Map<String, ChamberState> chamberStates = new ConcurrentHashMap<>();
public ChamberState getChamberState(String chamberId) {
    return chamberStates.computeIfAbsent(chamberId, key -> new ChamberState());
}
// EventLogger.java: thread-safe synchronized list
private final List<String> sensorLogs = Collections.synchronizedList(new ArrayList<>());
```

Issue 7: Nested Locks Deadlock Hazard & Subclassing Thread Directly (Lines 208-240, 275-284)

SE301 Syllabus Category: Week 7 Multithreading (Runnable & ExecutorService) | Week 11 Deadlocks (Avoid Holding 2 Locks Simultaneously)

1. LEGACY CODE (SVFRMS.java)

```
public static void archiveAlerts() {
    synchronized (alertLock) {
        if (!alerts.isEmpty()) { toArchive.addAll(alerts); alerts.clear(); totalAlerts = 0; }
        synchronized (reportLock) { archivedAlerts.addAll(toArchive); reports.add("Archived " + toArchive.size() + " alerts"); }
    }
}
static class ChamberThread extends Thread { ... }
```

2. Issues with the Legacy Code:

- Nested Locks Deadlock Hazard (Week 11 Slide 12): Both generateDailyReport() and archiveAlerts() hold two locks at the same time (synchronized(alertLock) -> synchronized(reportLock)), risking circular-wait deadlock if lock acquisition order ever diverges.
- Inheritance over Composition (Week 7): ChamberThread and ControlThread extend Thread directly instead of implementing Runnable.

3. How It Was Fixed:

Separated lock acquisitions in ReportGenerator so ALERT_LOCK is acquired and released inside AlertManager before REPORT_LOCK is acquired. Replaced Thread subclasses with Runnable tasks (ChamberSensorTask, ControlWorker) executed via Executors.newFixedThreadPool(4):

REFACTORED CODE (How It Was Fixed)

```
// ReportGenerator.java: acquires & releases ALERT_LOCK first, then locks REPORT_LOCK separately (no nested locks)
public void archiveAlerts() {
    int archivedCount = alertManager.archiveActiveAlerts(); // Locks & unlocks ALERT_LOCK internally
    reportLock.lock();
    try { reports.add(archivedCount > 0 ? "Archived " + archivedCount + " alerts" : "No alerts to archive"); }
    finally { reportLock.unlock(); }
}
```

3. PROPOSED DESIGN & JUSTIFICATION

Table 1 summarizes our refactored package structure. Every class created is strictly necessary, has a single well-defined responsibility, and preserves 100% of existing behavior while eliminating all identified flaws:

Package & Classes	Key Refactoring Changes	Justification & SE301 Concept
org.example.model ThresholdRange, CropProfile, ChamberState, ChamberSummary, ControlRequest	Encapsulated domain models with private fields, bounded adjustment methods on ChamberState, ReentrantReadWriteLock on ChamberState/ChamberSummary, immutable ControlRequest DTO, and CropProfile (with Builder) & ThresholdRange.	Enforces Information Hiding, eliminates Feature Envy and magic numbers, and provides thread-safe reads/writes (Week 10 ReadWriteLocks).
org.example.concurrency LockFactory	Centralized thread-safe LockFactory.getLock(String) managing named ReentrantLock instances (ALERT_LOCK, REPORT_LOCK, RESOURCE_LOCK).	Directly applies Week 10 LockFactory (Singleton + Factory) to replace exposed static Object locks.
org.example.crop CropRegistry	ConcurrentHashMap<String, CropProfile> repository pre-loaded with the 6 legacy crops and exposing registerCrop() and removeCrop().	Applies Repository Pattern (Week 2) and OCP so crops are added/removed without editing sensor code.
org.example.sensor SensorProcessor, SensorService, ChamberSensorTask	SensorProcessor strategy interface + 5 registered processors (Temperature, Humidity, Soil, Light, Nutrient) in SensorService; ChamberSensorTask implements Runnable.	Applies Strategy Pattern, OCP, DIP, DRY, and Week 7 Multithreading (Runnable + ExecutorService).
org.example.control ControlDevice, ControlService, ControlWorker	ControlDevice strategy interface + 6 registered actuators and bounded Producer-Consumer queue in ControlService; ControlWorker implements Runnable.	Replaces 6-branch if-else chain in activateControl() with Strategy Pattern & safe queue synchronization.
org.example.service EventLogger, AlertManager, ChamberManager, ResourceMonitor, ReportGenerator	5 single-responsibility services replacing handleEvent() and static state using synchronizedList, ALERT_LOCK, ConcurrentHashMap.computeIfAbsent, RESOURCE_LOCK, and separated REPORT_LOCK.	Eliminates Selector Arguments, satisfies SRP/ISP, fixes check-then-act and += races, and prevents deadlocks (Week 11).

4. FINAL ARCHITECTURE

4.1 High-Level Architectural & Component Diagram

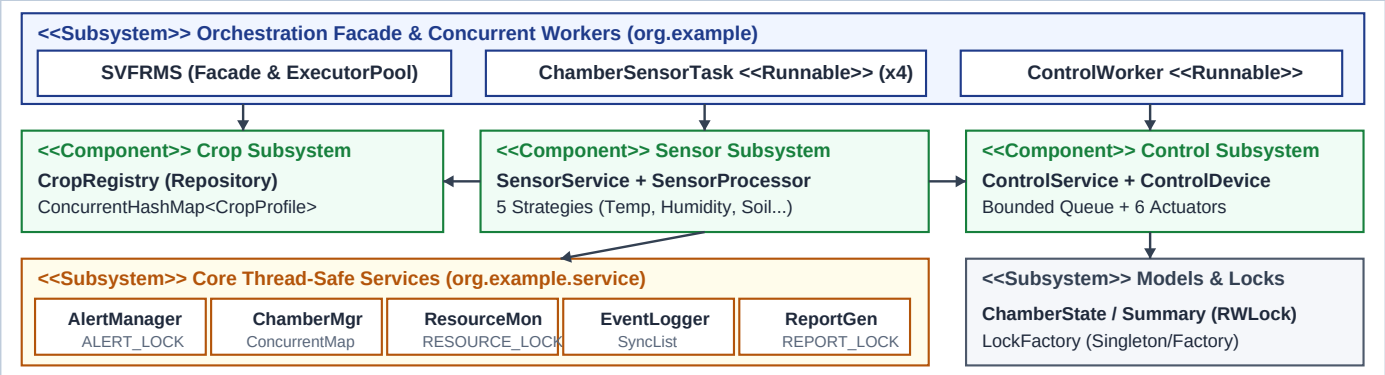

Figure 1: High-Level Architectural & Component Diagram of the Refactored SVFRMS System

4.2 UML Class Diagram

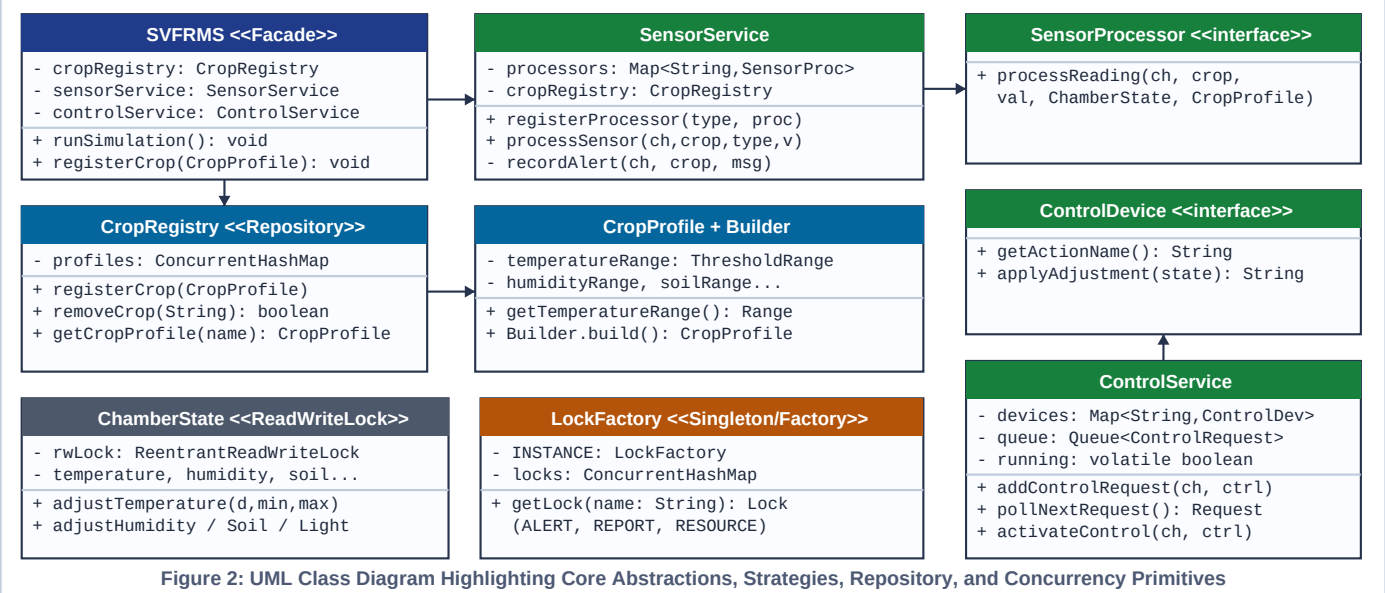

Figure 2: UML Class Diagram Highlighting Core Abstractions, Strategies, Repository, and Concurrency Primitives

4.3 High-Level UML Sequence Diagram

Figure 3 traces the concurrent runtime interaction when a ChamberSensorTask detects an out-of-range reading, records an alert, enqueues a ControlRequest into ControlService, and is asynchronously mitigated by ControlWorker.

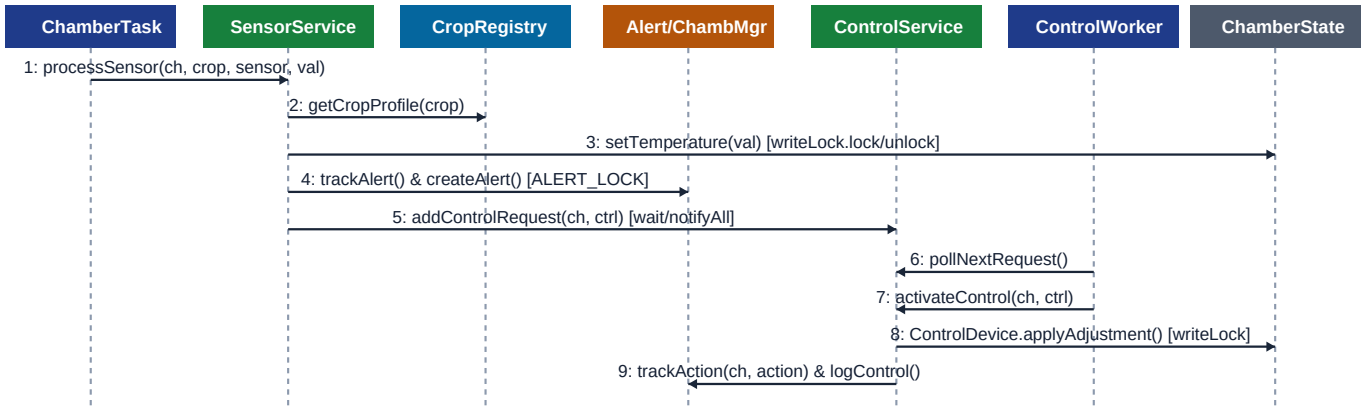

Figure 3: UML Sequence Diagram for Sensor Evaluation, Alert Creation, and Producer-Consumer Control Actuation

5. EXTENSIBILITY AND MAINTAINABILITY

A core requirement of Specification Section 4.5 is demonstrating how our proposed design supports future functional changes-adding new crops, removing existing crops, or adding/removing sensors and actuators-with minimal impact on the rest of the system in accordance with the Open-Closed Principle (OCP) and Single Responsibility Principle (SRP).

5.1 Adding New Crops (e.g., Kale, Bell Pepper, Bok Choy, Eggplant)

In the legacy system, adding four new crops (Kale, Bell Pepper, Bok Choy, and Eggplant) required modifying the monolithic processSensor() method in three separate places (Temperature, Humidity, and Soil branches), adding up to 24 new if-else blocks and risking regression bugs in existing crop logic.

In our refactored design, all crop-specific thresholds and resource consumption parameters are encapsulated inside CropProfile data objects managed by CropRegistry. Adding Kale, Bell Pepper, Bok Choy, and Eggplant requires ZERO changes to SensorService, ControlService, or SVFRMS-we simply build and register the new profiles via CropProfile.Builder:

```
// Adding Kale, Bell Pepper, Bok Choy, and Eggplant with zero changes to SensorService (OCP):
system.registerCrop(new CropProfile.Builder("Kale")
        .temperatureRange(15, 22).humidityRange(60, 75).soilMoistureRange(50, Double.MAX_VALUE)
        .tempEnergyUsage(10, 15).lowHumidityWaterUsage(20).lowSoilWaterUsage(40).build());

system.registerCrop(new CropProfile.Builder("Bell Pepper")
        .temperatureRange(20, 28).humidityRange(60, 75).soilMoistureRange(50, Double.MAX_VALUE)
        .tempEnergyUsage(10, 15).lowHumidityWaterUsage(20).lowSoilWaterUsage(45).build());

system.registerCrop(new CropProfile.Builder("Bok Choy")
        .temperatureRange(15, 22).humidityRange(65, 80).soilMoistureRange(55, Double.MAX_VALUE)
        .tempEnergyUsage(10, 15).lowHumidityWaterUsage(20).lowSoilWaterUsage(40).build());

system.registerCrop(new CropProfile.Builder("Eggplant")
        .temperatureRange(22, 30).humidityRange(60, 80).soilMoistureRange(50, Double.MAX_VALUE)
        .tempEnergyUsage(10, 15).lowHumidityWaterUsage(20).lowSoilWaterUsage(50).build());
```

Because SensorService evaluates every sensor reading polymorphically against profile.getTemperatureRange(), profile.getHumidityRange(), profile.getSoilMoistureRange(), profile.getLightLevelRange(), and profile.getNutrientLevelRange(), any newly registered crop is automatically supported across all five sensor types.

5.2 Removing Existing Crops & Extending Sensors / Control Devices

- Removing an Existing Crop: Decommissioning a crop such as Cucumber or Spinach requires no edits to SensorService; a single thread-safe repository call system.removeCrop("Cucumber") removes its profile from CropRegistry's ConcurrentHashMap.
- Adding/Removing Control Actuators: ControlService stores ControlDevice strategies in a ConcurrentHashMap. A new actuator (e.g., 'Heating Unit') is added via controlService.registerDevice("Heating Unit", strategy) and removed via controlService.removeDevice("Grow Light").
- Adding/Removing Sensor Types: SensorService maps sensor names to SensorProcessor strategies. A new 'CO2' or 'pH' sensor processor is added via sensorService.registerProcessor("CO2", processor) without modifying existing processors.

6. DESIGN PATTERNS

Every design pattern applied in our refactored architecture was selected directly from the SE301 lecture syllabus (Week 2 Design Patterns and Week 10 Explicit Locks) to resolve a specific structural or behavioral flaw in the legacy implementation:

Design Pattern & Category	Refactored Classes	Legacy Problem Solved	Why Selected & Design Benefit
1. Strategy Pattern (GoF Behavioral - Wk 2)	SensorProcessor & SensorService; ControlDevice & ControlService	processSensor() and activateControl() used sprawling if-else chains over sensor type and control device strings.	Encapsulates each sensor evaluation rule and actuator behavior behind an interface, enabling runtime extension/removal (OCP & DIP).
2. Builder Pattern (GoF Creational - Wk 2)	CropProfile & CropProfile.Builder	Crop thresholds and resource increments were scattered as magic numbers across nested conditionals; some crops omitted humidity/soil rules.	Constructs immutable CropProfile objects step-by-step with optional ranges and sensible defaults (light min=500, nutrient min=40).
3. Singleton + Factory (GoF Creational - Wk 2 & Wk 10 Locks)	LockFactory	SVFRMS exposed four public static Object lock fields (alertLock, reportLock, waterPumpLock, nutrientLock), violating encapsulation.	Directly implements Week 10 LockFactory.getLock(String) using a private Singleton constructor and ConcurrentHashMap.computeIfAbsent.
4. Repository Pattern (PoEAA Architectural - Wk 2)	CropRegistry	No centralized domain store existed for crop metadata; crop rules were hardcoded inside procedural sensor logic.	Provides a collection-like abstraction over CropProfile domain objects backed by ConcurrentHashMap.
5. Facade & Service Layer (GoF Structural / PoEAA - Wk 2)	SVFRMS (Facade); AlertManager, ChamberManager, ResourceMonitor, EventLogger, ReportGenerator	The God Class SVFRMS and 6-flag handleEvent() method tightly coupled all subsystems and violated SRP/ISP.	SVFRMS provides a clean entry point delegating work to five single-responsibility services wired via Dependency Injection.
6. Producer-Consumer (Concurrency - Wks 7 & 9)	ChamberSensorTask, ControlService (Bounded Queue), ControlWorker	controlQueue encoded requests as raw 'chamber|control' strings split via regex, and addControlRequest() risked hanging producers.	Decouples sensor producer tasks from actuator execution using immutable ControlRequest DTOs and safe wait()/notifyAll() guards.

6.1 Summary of Concurrency & Thread-Safety Improvements (Weeks 7-11)

Table 3 summarizes how each concurrency mechanism from Weeks 7-11 eliminates the race conditions and deadlock hazards of the legacy code while preserving identical functional behavior:

Shared State / Workflow	Legacy Concurrency Defect	Refactored Thread-Safe Mechanism (SE301 Syllabus)
Active & Archived Alerts (AlertManager)	handleEvent("CREATE_ALERT") mutated alerts and totalAlerts++ without locking, racing with archiveAlerts().	Protected all alert mutations, archival, and snapshot reads with LockFactory.getLock(ALERT_LOCK) in try/finally blocks (Week 10 Locks).
Water & Energy Counters (ResourceMonitor)	Unprotected non-atomic totalWaterUsage += ... and totalEnergyUsage += ... across 4 chamber threads; unused waterPumpLock / nutrientLock.	Protected all resource increments and reads under LockFactory.getLock(RESOURCE_LOCK), eliminating lost updates (Weeks 9-10).
Chamber Maps (ChamberManager)	Raw HashMap with check-then-act race condition: if (!map.containsKey(k)) map.put(k, new ...).	Replaced with ConcurrentHashMap and atomic computeIfAbsent(chamberId, k -> new ...) (Week 9 ConcurrentHashMap).
Per-Chamber State & Summary (ChamberState, ChamberSummary)	Unprotected concurrent field reads/writes between ChamberThread and ControlThread.	Encapsulated each model with ReentrantReadWriteLock (readLock() for getters/formatting, writeLock() for adjustments, Week 10).
Sensor Logs (EventLogger)	Raw ArrayList appended concurrently by 4 sensor threads without synchronization.	Wrapped with Collections.synchronizedList(new ArrayList<>()) and copied under synchronization for reporting (Week 9).
Reporting & Alert Archiving (ReportGenerator)	archiveAlerts() and generateDailyReport() acquired nested locks synchronized(alertLock) { synchronized(reportLock) }.	Eliminated nested locks (Week 11: 'Don't hold 2 locks at the same time') by acquiring thread-safe snapshots first and locking REPORT_LOCK separately.
Thread Management (SVFRMS, ChamberSensorTask, ControlWorker)	Subclassed Thread directly (extends Thread) and manually started/joined 4 ChamberThread instances.	Implemented Runnable tasks and managed chamber execution via Executors.newFixedThreadPool(4) (Week 7 Multithreading).

Verification of Existing Behavior Preservation:

Executing 'java -jar run.jar' runs all 4 chambers (CH1 Tomato, CH2 Strawberry, CH3 Lettuce, CH4 Chili) across 20 random sensor iterations, prints all [ALERT] and green ANSI [CONTROL] logs, outputs '===== DAILY REPORT =====', and writes 'archive/daily_report.txt' in the exact legacy format with zero race conditions or deadlocks.